

# Gemini Historical Artifact Description

## Project Documentation

Generated: June 24, 2026

## Table of Contents

1. Project Overview
2. Features
3. Requirements
4. Installation & Setup
5. Configuration
6. Usage Guide
7. Project Structure
8. Docker Deployment
9. API Reference
10. Troubleshooting

# 1. Project Overview

The **Gemini Historical Artifact Description** is a web application built with Streamlit that leverages Google's Gemini AI model to generate well-structured blog posts. Users can provide a topic, upload an image, or both, and the application uses artificial intelligence to generate high-quality content.

This tool is particularly useful for content creators, bloggers, and researchers who want to quickly generate initial drafts for blog posts, enhance content with AI assistance, or analyze images and create descriptive content.

## 2. Features

- **Multi-input Support:** Accept blog topics (text) and/or images (JPG, PNG, JPEG)
- **AI-Powered Generation:** Uses Google's Gemini 2.5-flash model for intelligent content creation
- **Customizable Length:** Slider to control blog post length (200-2000 words)
- **Image Display:** Show uploaded images alongside generated content
- **Content Editing:** Built-in text area for users to edit generated content
- **Download Capability:** Export generated blogs as plain text files
- **Error Handling:** Comprehensive error messages for user guidance
- **Responsive Design:** Wide layout for optimal viewing experience

## 3. Requirements

### System Requirements:

- Python 3.11 or higher
- Docker (optional, for containerized deployment)
- Internet connection (for Google Gemini API access)
- Valid Google Gemini API key

### Python Dependencies:

```
streamlit
python-dotenv
Pillow
google-genai
```

## 4. Installation & Setup

### ***Step 1: Clone or Download the Project***

Clone the repository or download the project files to your local machine.

### ***Step 2: Create a Virtual Environment (Recommended)***

```
python -m venv venv
```

For Windows:

```
venv\Scripts\activate
```

For macOS/Linux:

```
source venv/bin/activate
```

### ***Step 3: Install Dependencies***

```
pip install -r requirements.txt
```

### ***Step 4: Configure API Key***

Create a .env file in the project root directory with the following content:

```
GEMINI_API_KEY=your_actual_api_key_here
```

## 5. Configuration

### **Environment Variables**

The application requires the following environment variable to be set:

#### **GEMINI\_API\_KEY**

Your Google Gemini API key. Obtain this from the [Google AI Studio](https://aistudio.google.com) (<https://aistudio.google.com>).

### **Streamlit Configuration**

The application is configured with the following Streamlit settings:

- **Page Title:** Multimodal AI Blog Generator
- **Layout:** Wide
- **Model Used:** gemini-2.5-flash

## 6. Usage Guide

### *Running the Application*

```
streamlit run app.py
```

The application will open in your default browser at <http://localhost:8501>

### *Using the Application*

1. **Provide Input:** Enter a blog topic in the text input field and/or upload an image
2. **Adjust Length:** Use the slider to set the desired blog post length (200-2000 words)
3. **Generate:** Click the 'Generate Blog' button to create the content
4. **Edit:** Review and edit the generated content in the text area if needed
5. **Download:** Click 'Download Blog' to export the content as a text file

### *Input Combinations*

The application supports three input modes:

- **Topic Only:** Generate a blog post based on the provided topic
- **Image Only:** Analyze the image and generate descriptive content
- **Topic + Image:** Create a blog post that incorporates both the topic and image content

## 7. Project Structure

```
artifact/  
■■■ app.py # Main Streamlit application  
■■■ requirements.txt # Python dependencies  
■■■ Dockerfile # Docker configuration  
■■■ .env # Environment variables (create this file)  
■■■ .gitignore # Git ignore file
```

## 8. Docker Deployment

### *Building the Docker Image*

```
docker build -t gemini-blog-generator .
```

### *Running the Container*

```
docker run -p 8501:8501 --env-file .env gemini-blog-generator
```

The application will be accessible at <http://localhost:8501>

### Docker Configuration Details

- **Base Image:** python:3.11-slim
- **Working Directory:** /app
- **Port:** 8501
- **System Dependencies:** build-essential, libjpeg-dev, zlib1g-dev, libpng-dev

## 9. API Reference

### *Google Gemini API*

The application uses the **google-genai** library to interact with Google's Gemini API.

### Key Functions

- **genai.Client():** Initializes the API client with the provided API key
- **client.models.generate\_content():** Generates content using the specified model and input

### Model Used

**gemini-2.5-flash:** A fast, multimodal model suitable for text and image analysis

## 10. Troubleshooting

### 1. API Key Error

Problem: Error: Invalid API key or API key not found

Solution: Ensure the .env file exists in the project root with the correct GEMINI\_API\_KEY value. Verify the API key is valid at Google AI Studio.

### 2. Image Upload Issues

Problem: Image fails to upload or generate content from image

Solution: Ensure the image format is JPG, JPEG, or PNG. Check file size is reasonable. For very large images, try compressing them first.

### 3. Module Import Errors

Problem: ModuleNotFoundError for streamlit, PIL, or google-genai

Solution: Ensure virtual environment is activated and dependencies are installed: pip install -r requirements.txt

### 4. Port Already in Use

Problem: Port 8501 is already in use

Solution: Kill the existing process or specify a different port: streamlit run app.py --server.port 8502

### 5. Connection Issues

Problem: Cannot connect to Google Gemini API

Solution: Check internet connection. Verify API key is active. Check rate limits haven't been exceeded.

